

# Variables and console input

CSC103 Programming Fundamentals / Lecture 04

**Dr. Muhammad Farhan**

Associate Professor of Computer Science  
Department of Computer Science  
COMSATS University Islamabad  
Sahiwal Campus

Fall 2026

# The problem for this lecture

Read a quantity and price using Scanner. Print their product. Then read a one-letter category. Use quantity 3, price 12.5, category A.

Create one Scanner and read quantity, price and category in order.

# Learning objectives

- Declare and initialise variables and constants
- Read numeric and character input
- Trace assignment as a change of stored value

CLO-1

Variables and initialisation

Assignment copies a value

Scanner input

Reading one character

# Variables and initialisation

- A variable has a declared type and a name.
- Initialisation supplies its first value. A local variable must receive a value before a read.

## Example

```
int seats = 30;  
seats = 28;  
final int ROOM_LIMIT = 40;
```

# Assignment copies a value

- The right-hand expression is evaluated before assignment.
- For primitive variables, assigning one variable to another copies the current primitive value.

## Example

```
int a = 7;  
int b = a;  
a = 9;  
// b is still 7
```

# Scanner input

- Create one Scanner for console input.
- `nextInt` reads an integer token and `nextDouble` reads a decimal token.
- The entered token must match the requested type.

## Example

```
java.util.Scanner in =  
    new java.util.Scanner(System.in);  
int age = in.nextInt();
```

# Reading one character

- Scanner has no nextChar method.
- next().charAt(0) reads the first UTF-16 code unit of the next token.
- For this exercise, input is one basic Latin character.

## Example

```
String token = in.next();  
char choice = token.charAt(0);
```

# Choosing a type from the data

- A quantity that counts whole items fits an integer type.
- A price with a fractional part needs a suitable numeric representation.
- For introductory arithmetic, double is convenient, but exact financial applications need decimal handling.

## Example

```
quantity: 3
price: 12.5
category: A
Each input has a different role and type.
```



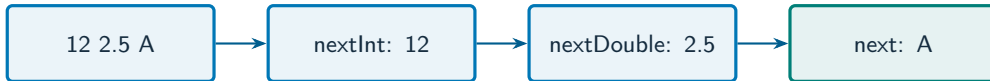
# Console input is a sequence of tokens

- Scanner reads the next available token when a next method succeeds.
- The program must request the token type in the order the user supplies it.
- A character read through `next().charAt(0)` uses only the first character of that token.

## Example

```
Input tokens: 3 12.5 A
nextInt consumes 3
nextDouble consumes 12.5
next consumes A
```

# Input tokens become typed values



## What to notice

Each Scanner call consumes the next token according to the requested type.

# Key distinctions

| Declaration          | Initial value | May be reassigned?          |
|----------------------|---------------|-----------------------------|
| int quantity = 3     | 3             | Yes                         |
| double price = 12.5  | 12.5          | Yes                         |
| char category = 'A'  | A             | Yes                         |
| final int LIMIT = 20 | 20            | No after initial assignment |

These distinctions determine which operation or control structure fits the problem.

# First example: execution

```
int initial = 10;  
int remaining = initial;  
remaining = remaining - 3;  
final int CAPACITY = 20;  
System.out.println(remaining);  
System.out.println(initial);
```

# First example: result

7

10

Assigning remaining does not change initial.

Scanner input

Reading one character

# Guided practice

Read a quantity and price using Scanner. Print their product. Then read a one-letter category. Use quantity 3, price 12.5, category A.

Attempt the solution before continuing.  
Use the definitions and examples from this lecture.

# Solution strategy

- Create one Scanner and read quantity, price and category in order.
- Calculate quantity \* price using the numeric values.
- Print the product and the selected category.

Sample console input  
3 12.5 A

# Complete solution

```
public class Solution04 {  
    public static void main(String[] args) throws Exception {  
        java.util.Scanner in = new java.util.Scanner(System.in);  
        in.useLocale(java.util.Locale.US);  
        int quantity = in.nextInt();  
        double price = in.nextDouble();  
        char category = in.next().charAt(0);  
        double total = quantity * price;  
        System.out.println(total);  
        System.out.println(category);  
    }  
}
```



# Execution trace

| Read or calculation | Consumed token | Stored result  |
|---------------------|----------------|----------------|
| nextInt()           | 3              | quantity = 3   |
| nextDouble()        | 12.5           | price = 12.5   |
| next().charAt(0)    | A              | category = 'A' |
| quantity * price    | No new input   | total = 37.5   |

Follow the changing state in execution order.

# Solution result

37.5  
A

Why this result follows  
Print the product and the selected  
category.

# A common incorrect approach

```
int total;  
System.out.println(total);
```

## Example

A local variable has no usable automatic initial value. Assign total before printing it, **for**  
example **int** total = 0.

# Boundary and failure checks

Check the stated input assumptions.  
Read a rectangle length and width. Print area and perimeter. State the units and allowed input range.

Use `int quantity = in.nextInt()`,  
`double price = in.nextDouble()`, and  
`char category = in.next().charAt(0)`.  
Product: 37.5. Category: A.

# Check your understanding

After `int x=4; int y=x; y=8;`, what is `x`? Can a final variable be assigned again?

Write a prediction and explain the rule that supports it.

# Answer and explanation

x remains 4. A final variable cannot be reassigned after its initial assignment.

A local variable has no usable automatic initial value. Assign total before printing it, for example `int total = 0`.

# Compare cases and predict outcomes

| Next token | Requested operation           | Result           |
|------------|-------------------------------|------------------|
| 12         | <code>nextInt()</code>        | int value 12     |
| 2.5        | <code>nextDouble()</code>     | double value 2.5 |
| A          | <code>next().charAt(0)</code> | char value A     |

## Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

# Apply the idea

## Independent practice

A shop sells 12 items at 2.5 each. Show the variable declarations and total. Then predict what happens if `nextInt()` is used for the token 2.5.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.



# Solution and reasoning

```
int quantity = 12;  
double unitPrice = 2.5;  
double total = quantity * unitPrice;  
System.out.println(total);
```

- The int operand is promoted for multiplication with double.
- The result is 30.0.
- Scanner.nextInt() cannot consume 2.5 as an int and throws InputMismatchException.

# Lecture summary

- and initialise variables and constants
  - numeric and character input
  - assignment as a change of stored value
- Create one Scanner and read quantity, price and category in order.
  - Calculate quantity \* price using the numeric values.
  - Print the product and the selected category.

# After-class practice

Read a rectangle length and width. Print area and perimeter. State the units and allowed input range.

Syllabus reading

Liang Ch 2

Next lecture: Types and arithmetic expressions